

Final Report:

Student Course Management System

Institute:	TechPanda Academy
Student:	B Shankar Ganesh
Course:	Data Engineering (Weekend Batch)
Project:	Student Course Management System

1. Mini Project Overview

The Student Course Management System is a relational database designed to track and manage the core academic activities of a university. The objective of this project is to implement a robust data architecture that handles student records, department distributions, course catalogs, faculty assignments, and enrollment histories. By utilizing Advanced SQL techniques including complex Joins, Correlated Subqueries, Aggregations, and Window Functions this system transforms raw academic data into actionable insights to support academic administration and monitor student success.

2. Database schema

The database consists of five normalized tables designed with strict data integrity constraints to prevent orphan records and bad data entry.

- **Students:** Stores demographic and contact data.
 - *Primary Key:* student_id
 - *Constraints:* NOT NULL applied to names and dates of birth. A CHECK constraint ensures the gender column only accepts 'M' or 'F'. The email column enforces UNIQUE constraints to prevent duplicate user accounts.
- **Departments:** Acts as the central hub for categorizing both courses and faculty.
 - *Primary Key:* dept_id
 - *Constraints:* dept_name is UNIQUE and NOT NULL.
- **Courses:** Catalogs the classes offered by the university.
 - *Primary Key:* course_id
 - *Foreign Key:* dept_id (References Departments)
 - *Constraints:* A CHECK constraint ensures course credits remain logically bounded between 1 and 5.
- **Faculty:** Records university teachers and their departmental affiliations.
 - *Primary Key:* faculty_id
 - *Foreign Key:* dept_id (References Departments)
- **Enrollments:** The associative (junction) table resolving the many-to-many relationship between Students and Courses.
 - *Primary Key:* enroll_id
 - *Foreign Keys:* student_id (References Students), course_id (References Courses)
 - *Constraints:* A CHECK constraint restricts the grade column to a standard 0.0 to 10.0 scale, preventing typographical errors during data entry.

3. Key queries

To extract meaningful data and fulfill the project requirements, several advanced SQL queries were engineered. Below are the highlights of the system's analytical capabilities:

Algorithmic Seniority and Mentorship (Self-Join) To establish a mentorship hierarchy without altering the original table schema to include a dedicated mentor column, a logical business rule was applied using a Self-Join.

SQL

```
SELECT mentee.name AS Junior_Faculty, mentor.name AS Mentor
FROM Faculty mentee
JOIN Faculty mentor
    ON mentee.dept_id = mentor.dept_id
    AND mentee.faculty_id > mentor.faculty_id;
```

Purpose: By joining the Faculty table to itself on the condition that the mentor has a lower faculty_id (indicating earlier tenure) within the same department, the system organically maps senior faculty as mentors to junior faculty.

Weighted GPA Calculation (Aggregation & Math) Calculating a student's GPA requires weighting the grade against the course credits, rather than taking a simple average.

SQL

```
SELECT e.student_id, SUM(e.grade*c.credits)/SUM(c.credits) AS Calculated_GPA
FROM Enrollments e
JOIN Courses c ON e.course_id = c.course_id
WHERE e.grade IS NOT NULL
GROUP BY e.student_id;
```

Purpose: Using arithmetic operators combined with the SUM() aggregate function, this query accurately reflects academic standing while safely filtering out active courses where the grade is not yet assigned.

Grade Distribution (Window Functions) Understanding how a student performs relative to their peers in a specific class is critical for grading curves.

SQL

```
SELECT student_id, course_id, grade,  
       RANK() OVER(PARTITION BY course_id ORDER BY grade DESC) AS grade_rank,  
       DENSE_RANK() OVER(PARTITION BY course_id ORDER BY grade DESC) AS dense_grade_rank  
FROM Enrollments  
WHERE grade IS NOT NULL;
```

Purpose: By partitioning the data by course_id, the system resets the ranking sequence for every new class. Using both RANK and DENSE_RANK allows the administration to see how the database handles tied scores without collapsing subsequent ranking numbers.

4. Insights (e.g., top-performing students, department statistics)

By querying the populated dataset (10 students, 5 courses, 3 departments, and 15 enrollment records), the following insights were generated:

- **Top-Performing Students:** The data reveals that Diana Prince holds the highest raw grade (9.5 in Quantum Mechanics). However, Alice Johnson demonstrates the highest sustained academic performance with a heavy course load, achieving a 9.0 and 8.5 in Fall 2023, followed by a 9.2 in Spring 2024.
- **Department Statistics and Load:** The Computer Science department (Dept 1) is currently bearing the heaviest systemic load. It hosts 60% of the available courses (Data Structures, Algorithms, Database Systems) and holds the vast majority of student enrollments. Conversely, the Mathematics department has a smaller footprint with only one course mapped to it.
- **Course Popularity:** Course 101 (Data Structures) is the most enrolled-in class in the database across all semesters, indicating it requires the most logistical attention and potential faculty expansion for upcoming terms.
- **Data Integrity Verification:** During testing, the CHECK constraints successfully functioned as automated gatekeepers. The system inherently rejects impossible grades and stops the input of unrecognized gender formats, ensuring the administrative data remains secure against fundamental data-entry errors.